

MiniLang Compiler Lab Report

Student Name: Md Arzan Molla | **Roll Number:** 241030042 | **Assigned Variant:** Variant 2 (Structs & Nested Functions with Static Scoping)

1. Design Decisions

For this lab assignment, I had to build a working compiler front-end and back-end for MiniLang. My roll number is 241030042, so $241030042 \% 4 = 2$. This meant I was assigned Variant 2, which requires user-defined structs (records) and nested functions with static scoping.

1.1 Hand-written Lexer (DFA)

Our teacher asked us not to use Lex or Flex. I wrote the lexer from scratch in `src/lexer.c` using a basic DFA approach. It reads the source code character by character and tracks line and column numbers. If an invalid symbol or unclosed string literal occurs, it prints the exact line and column position. For keywords, since my roll ends in 42, I reversed the normal English words and added 42 at the end (for example, struct becomes tcurts42, function becomes noitcnuf42, let becomes tel42, and print becomes tnirp42).

1.2 Recursive-Descent Parser & AST

I chose recursive-descent parsing because it is straightforward to write by hand. To avoid infinite loops from left recursion in arithmetic expressions (like $E \rightarrow E + T$), I split the grammar rules into tail-recursive form ($E \rightarrow T E'$ and $E' \rightarrow + T E' \mid \epsilon$). For if-statements, I used left-factoring on the optional else part to make the grammar LL(1) without backtracking. The parser builds an Abstract Syntax Tree (AST) using C tagged unions in `src/ast.c` to represent declarations, statements, struct types, and nested functions.

1.3 Symbol Table and Static Scoping

Since Variant 2 requires nested functions, the compiler must support static scoping. I implemented the symbol table as a tree of scopes in `src/symbol_table.c`. Each Scope struct stores symbols for its local block and has a parent pointer pointing to the outer function's scope. When an inner function looks up a variable, it first checks its own scope, and if missing, walks up the parent pointer chain. In `src/semantic.c`, I wrote checks for 5 distinct semantic errors: (1) Undeclared variables/functions, (2) Duplicate declarations in the same scope, (3) Type mismatch in assignment, (4) Non-existent struct field access, and (5) Function return type mismatch.

1.4 Three-Address Code (TAC) and Optimizations

The AST is linearized into Three-Address Code (TAC) in `src/tac.c` using temporary variables (`t1`, `t2`, ...) and labels. On top of TAC, I wrote two optimizations in `src/optimizer.c`: (1) **Constant Folding**: evaluates constant operations like $10 + 20$ into 30 at compile time. (2) **Dead Code Elimination**: removes unreachable instructions after return or goto statements and cleans up unused temporaries.

1.5 Backend and Virtual Machine

The compiler translates TAC into stack-machine bytecode (`PUSH_INT`, `STORE_VAR`, `LOAD_VAR`, `ADD`, `CALL`, `RET`, `ALLOC_STRUCT`, `GET_FIELD`, `SET_FIELD`). A virtual machine runner in `src/vm.c` executes the bytecode with an activation stack and heap objects for struct records.

2. A Bug I Encountered and How I Debugged It

The Problem: When I tested nested functions with `test06_nested_functions.ml`, the compiler gave a semantic error saying an outer variable was 'undeclared' inside the nested function. Also, when calling functions with parameters, the values evaluated to 0 inside the function body.

Debugging Steps: I added debug print statements in `parser.c` and `semantic.c`. I discovered that: (1) My parser was storing nested function declarations in a separate list from regular statements, and my semantic analyzer was checking nested functions before outer local variables had been added to the symbol table. (2) In the stack machine, caller pushed arguments, but upon entry, the function did not pop them off the stack into local parameter variables.

The Fix: (1) I unified the function body so nested functions stay in the same linear statement list as other statements, allowing them to resolve outer variables via the parent scope link. (2) In `tac.c`, I added a `TAC_PARAM_POP` instruction that pops arguments in reverse order into local parameter variables on function entry. (3) I emitted a jump (`goto L_skip`) around nested functions to prevent

linear fall-through during outer execution. After this fix, both test06 and test10 passed completely.

3. One Thing I Would Change

If I had more time, the main thing I would change is the backend code generator. Currently it outputs stack-machine bytecode that runs in an interpreter. An interpreted VM is much slower than native code. If I were starting over with more time, I would write an x86-64 code generator that outputs actual GNU assembly (.s) files that could be assembled with gcc into real standalone binaries. I would also add a simple garbage collector for struct instances, since heap structs currently use calloc without explicit deallocation.